

04. CSS 視覺美學調校

一、關於 CSS

1.CSS 是什麼？

CSS (Cascading Style Sheets , 階層式樣式表) 則是用來設計網頁外觀的語言，可以把它想像成網頁的外觀與裝潢，負責控制網頁的視覺呈現，CSS 可以設定：

- 文字大小、字型與顏色
- 背景顏色或背景圖片
- 按鈕樣式
- 邊框、陰影、圓角效果
- 元素的位置與間距
- 響應式設計 (RWD)，讓網頁在手機、平板與電腦都能正常顯示
- 動畫與滑鼠互動效果



2. CSS 的基本語法

```
h1{
  color: blue;
  font-size: 36px;
}
```

組成：

```
選擇器 {
  屬性: 值;
}
```

例如：

```
p{  
  color:red;  
}
```

代表：

把所有 <p> 的文字改成紅色。

3. 常見選擇器 (Selector)

AI 最常使用的就是 Selector。

- 全域選擇器

```
*{  
}
```

作用：

控制HTML內所有標籤。

- 元素選擇器

```
h1{  
}
```

```
p{  
}
```

作用：

控制HTML內所有指定標籤。

- Class 選擇器 (最重要)

```
<div class="card">
```

```
.card{  
}
```

一個 Class 可以重複使用。

AI 最常產生的也是 Class。

- ID 選擇器

```
<div id="hero">
```

```
#hero{  
}
```

通常一頁只會出現一次。

目前大型專案已較少使用，Vibe Coding 也建議優先使用 Class。

4. 常用文字樣式

- color
- font-size
- font-weight
- font-family
- line-height
- letter-spacing
- text-align
- text-decoration
- text-transform

例如：

```
h2{
  font-size:32px;
  font-weight:bold;
  text-align:center;
}
```

5. 盒模型 (Box Model)

所有 HTML 元素都是一個盒子。

CSS 盒模型 (Box Model)

每個 HTML 元素都可以看作是一個矩形的盒子，由內而外包含：內容、內距、邊框、外距

margin (外距): 30px (左右), 30px (上下)

border (邊框): 5px (上下), 5px (左右)

padding (內距): 20px (上下), 20px (左右)

content (內容): 20px (寬度), 20px (高度)

這是盒子的內容區域 (寬度與高度)

盒模型尺寸計算 (以寬度為例)

總寬度 = margin-left + border-left + padding-left + width + padding-right + border-right + margin-right

高度計算方式與寬度相同 (使用 top 與 bottom)

範例程式碼

```
.box {
  width: 200px;
  height: 100px;

  padding: 20px;
  border: 5px solid #f0a500;
  margin: 30px;
  background-color: #e2f7e6;
}
```

- margin (外距)**: 盒子外部的透明區域，用於與其他元素分隔
- border (邊框)**: 圍繞在 padding 與 content 外圍的邊框
- padding (內距)**: 內容與邊框之間的透明區域
- content (內容)**: 實際的內容區域，顯示文字、圖片等

box-sizing 屬性

content-box (預設值)	width / height 只包含內容區域 總尺寸會因 padding、border 增加而變大
border-box	width / height 包含 padding 與 border 總尺寸固定為設定的值

四個重要屬性：

- margin
- padding
- border
- width / height

例如：

```
.card{  
padding:20px;  
margin:30px;  
}
```

6. 顏色設定

- color
- background
- background-color

例如：

```
background:#0f172a;  
color:white;
```

AI Prompt 常寫：

深色科技風

AI 就會大量使用背景色。

7. Flexbox 排版

目前 AI 最常使用的排版方式。

display:flex;

什麼是 Flexbox？

Flexbox 是一維排版工具，
可以輕鬆控制元素的排列方向、
對齊方式與間距，適用於各種
版面配置。



Flexbox 基本結構

在一個容器(Container)中，
直接子元素會自動成為彈性項目

重要屬性

justify-content

控制主軸(水平方向)對齊方式



align-items

控制交叉軸(垂直方向)對齊方式

gap

設定項目之間的間距



flex-wrap

控制換行方式

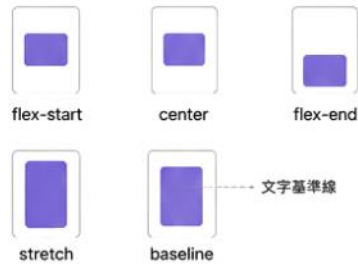


在一個容器(Container)中，
直接子元素會自動成為彈性項目
(Flex Item)



align-items

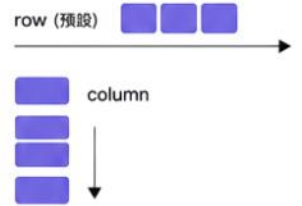
控制交叉軸(垂直方向)對齊方式



wrap

flex-direction

設定主軸方向



常見用途

水平排列



並排排列元素

垂直置中



在容器中垂直置中

卡片排版



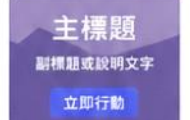
建立卡片式版面

導覽列

LOGO 首頁 服務 關於 聯絡

導覽列常用排版

Hero 區塊



置中內容與按鈕

範例程式碼

```
.container{
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  flex-wrap: wrap;
}
.item{
  width: 120px;
  height: 120px;
  background: #6366f1;
  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 10px;
}
```

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
  <div class="item">5</div>
</div>
```

執行效果



8. CSS Grid 網格排版系統

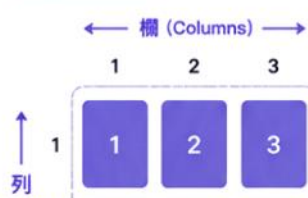
CSS Grid (★★★★★)

二維網格排版系統，適合複雜版面配置

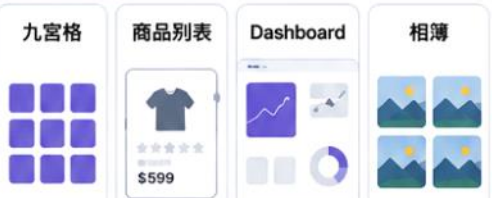
基本語法

```
.container{
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}
.item{
  background: #6366f1;
}
```

Grid 結構概念



適合製作



CSS Grid (★★★★★)

二維網格排版系統，適合複雜版面配置

基本語法

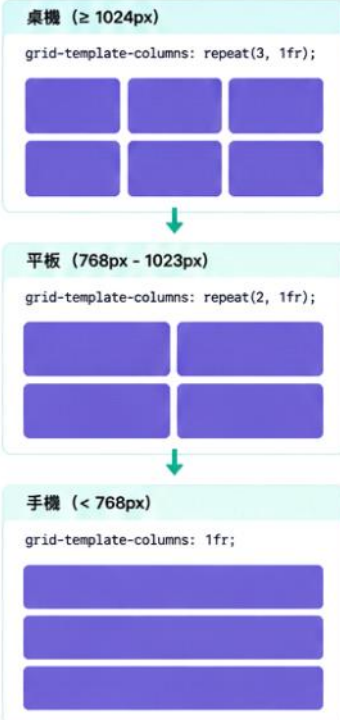
```
.container{
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.item{
  background: #6366f1;
  color: white;
  padding: 20px;
  text-align: center;
  border-radius: 8px;
}
```

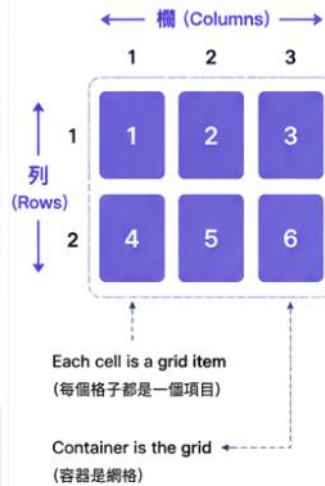
常用屬性

<code>display: grid</code>	啟用 Grid 布局
<code>grid-template-columns</code>	定義欄的數量與寬度
<code>grid-template-rows</code>	定義列的數量與高度
<code>gap</code>	設定格子之間的間距 (行與列)
<code>row-gap</code>	設定列之間的間距
<code>column-gap</code>	設定欄之間的間距
<code>justify-items</code>	設定內容在欄方向的對齊方式
<code>align-items</code>	設定內容在列方向的對齊方式
<code>grid-column</code>	設定項目跨越的欄範圍
<code>grid-row</code>	設定項目跨越的列範圍

響應式範例



Grid 結構概念



項目跨欄 / 跨列

跨兩欄

```
.item-a{
  grid-column: 1 / 3;
}
```



跨兩列

```
.item-b{
  grid-row: 1 / 3;
}
```

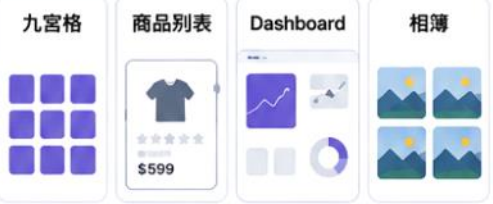


跨欄又跨列

```
.item-c{
  grid-column: 1 / 3;
  grid-row: 1 / 3;
}
```



適合製作

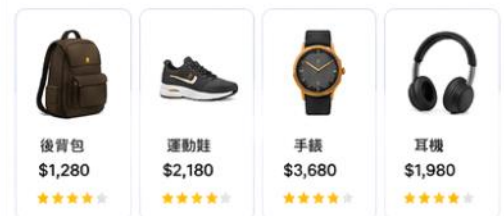


實際應用範例

1. 九宮格畫廊



2. 商品列表



3. Dashboard



4. 相簿範例



小技巧：

- ✓ 1fr 代表等分剩餘空間
- ✓ repeat (3, 1fr) 代表三等分

- ✓ gap 比使用 margin 更簡單

- ✓ 搭配 Media Query 打造 RWD 網格

AI 幾乎都會使用 Media Query。

例如：

```
@media (max-width:768px){  
}
```

RWD 響應式設計 (★★★★★)

讓網站在不同裝置上，都有最佳的瀏覽體驗

什麼是 RWD？



RWD (Responsive Web Design) 響應式網頁設計，透過 CSS 的 **Media Query**，讓網站能根據不同裝置的螢幕寬度，自動調整版面配置、字體大小、間距等，提供最佳的使用體驗。

常見斷點尺寸 (px)



Media Query 語法

```
/* 手機 (預設) */  
.container {  
  display: block;  
  padding: 16px;  
}  
  
/* 平板 */  
@media (max-width: 768px) {  
  .container {  
    padding: 24px;  
  }  
}  
  
/* 筆電 */  
@media (max-width: 1024px) {  
  .container {  
    max-width: 960px;  
    margin: 0 auto;  
  }  
}  
  
/* 桌機 */  
@media (min-width: 1200px) {  
  .container {  
    max-width: 1200px;  
  }  
}
```

手機優先
(Mobile First)
預設模式針對
手機設計

平板
(≤ 768px)
調整布局、
字體等

筆電
(≤ 1024px)
調整容器寬度、
間距

桌機
(≥ 1200px)
大螢幕優化顯示

同一個網站，在不同裝置上的呈現效果

手機 (≤ 480px)

平板 (≤ 768px)

筆電 (≤ 1024px)

桌機 (≥ 1200px)



常見調整項目

調整項目	手機 (≤ 480px)	平板 (≤ 768px)	筆電 (≤ 1024px)	桌機 (≥ 1200px)
容器寬度	100%	100%	960px	1200px
欄位數	1 欄	2 欄	2~3 欄	3~4 欄
字體大小	較小	中等	中等偏大	較大
間距 (padding/margin)	較小 (16px)	中等 (24px)	中等 (32px)	較大 (40px)
導覽列	漢堡選單	簡化選單	完整選單	完整選單 + 下拉選單
圖片縮放	100%	適當縮放	適當縮放	原始尺寸

RWD 設計技巧

- ✓ 採用 Mobile First 手機優先設計
- ✓ 使用彈性單位 (%，rem，vw，vh)
- ✓ 圖片使用 max-width: 100%
- ✓ 善用 Flexbox / Grid 排版
- ✓ 測試各種真實裝置
- ✓ 保持內容的可讀性與可用性



Prompt 撰寫建議

在 AI 工具中，可以這樣描述您的需求：

“支援 RWD 響應式設計，手機、平板、桌機皆正常顯示。使用 Flexbox 或 Grid 排版，並優化在不同裝置上的體驗。”



支援 RWD 響應式設計，手機、平板、桌機皆正常顯示。
使用 Flexbox 或 Grid 排版，並優化在不同裝置上的體驗。



記住：RWD 不只是讓網站「能看」，而是讓使用者「用得好」！

10. Hover 滑鼠滑入互動效果

Hover 效果

Hover 是滑鼠移到元素上時觸發的狀態，
可以讓網站更有互動性與專業感。

AI 很喜歡加入互動效果，讓網站更有吸引力！



基本語法

CSS

```
selector:hover {  
  /* 滑鼠移上去時的樣式 */  
}
```

說明：

當滑鼠移到指定元素上時，會套用大括號內的樣式。

簡單範例

```
button:hover {  
  transform: translateY(-5px);  
}
```

預設狀態

按鈕

Hover 狀態

按鈕

transform: translateY(-5px)

常見 Hover 效果範例

1. 上移效果

預設

按鈕

Hover

按鈕

```
button:hover {  
  transform: translateY(-5px);  
}
```

2. 放大效果

預設

按鈕

Hover

按鈕

```
button:hover {  
  transform: scale(1.05);  
}
```

3. 顏色變化

預設

按鈕

Hover

按鈕

```
button:hover {  
  background: #6366f1;  
  color: white;  
}
```

4. 陰影效果

預設

按鈕

Hover

按鈕

```
button:hover {  
  box-shadow: 0 8px 20px  
    rgba(0,0,0,0.15);  
}
```

5. 邊框效果

預設

按鈕

Hover

按鈕

```
button:hover {  
  border: 2px solid #6366f1;  
  color: #6366f1;  
}
```

6. 下劃線效果

預設

連結文字

Hover

連結文字

```
a:hover {  
  text-decoration: underline;  
  color: #6366f1;  
}
```

7. 透明度變化

預設



Hover



```
img:hover {  
  opacity: 0.8;  
}
```

8. 旋轉效果

預設



Hover



```
.star:hover {  
  transform: rotate(20deg)  
    scale(1.2);  
  color: #fbbf24;  
}
```

多個效果組合



卡片標題

這是一個卡片內容，滑鼠移上來會有多種效果組合！



卡片標題

這是一個卡片內容，滑鼠移上來會有多種效果組合！

```
.card {  
  transition: all 0.3s ease;  
}  
.card:hover {  
  transform: translateY(-8px) scale(1.03);  
  box-shadow: 0 15px 30px rgba(0,0,0,0.15);  
}
```

搭配 Transition 讓效果更順暢

搭配 transition 可以讓 Hover 效果產生平滑的動畫，
讓使用者體驗更好！

預設

按鈕

移入 0.3s

平滑動畫

Hover

按鈕

```
button {  
  transition: all 0.3s ease; /* 加上過渡效果 */  
}  
button:hover {  
  transform: translateY(-5px);  
  background: #6366f1;  
  box-shadow: 0 10px 20px rgba(99,102,241,0.3);  
}
```

常見應用場景

導覽列按鈕

商品卡片

運動鞋

CTA 按鈕

訂閱電子報，獲得最新資訊！

文字連結

常見應用場景

導覽列按鈕

首頁 服務 關於 聯絡

商品卡片

運動鞋 \$1,980 加入購物車

CTA 按鈕

訂閱電子報，獲得最新消息！ 立即訂閱

文字連結

查看更多文章 →

小技巧

✓ Hover 效果不建議過多，避免造成干擾

✓ 保持一致性，整站使用相同的互動邏輯

✓ 搭配 transition 效果更自然

✓ 在手機上，Hover 效果通常不會觸發

11. Transition 動畫

互動效果：Transition + Hover

讓元素在狀態改變時，產生平滑的動畫效果，提升使用者體驗！



基本概念

transition 屬性可以設定當元素的屬性發生變化時，要在多久時間內完成動畫效果。

CSS

```
.card{
  transition:0.3s; /* 動畫時間 0.3 秒 */
}
.card:hover{
  transform:scale(1.05); /* 滑鼠移上去時放大 */
}
```

預設狀態



卡片標題

這是一張卡片內容



Hover 狀態



卡片標題

這是一張卡片內容

範例：卡片放大效果

CSS

```
.card{
  width:200px;
  padding:20px;
  background:#6366f1;
  color:white;
  border-radius:12px;
  text-align:center;
  transition:.3s;
  box-shadow:0 4px 12px rgba(0,0,0,0.1);
}
.card:hover{
  transform:scale(1.05);
  box-shadow:0 8px 20px rgba(0,0,0,0.2);
}
```

效果展示

預設狀態



卡片標題

滑鼠移上去試試看



Hover 狀態



卡片標題

滑鼠移上去試試看



當滑鼠移上去時，卡片會在 0.3 秒內放大到 1.05 倍，並加深陰影，看起來更有層次感！

其他常見互動效果組合

上移效果



變亮效果



旋轉效果



邊框變色



透明度變化



 <pre>.box{ transition:.3s; } .box:hover{ transform: translateY(-5px); }</pre>	 <pre>.box{ transition:.3s; } .box:hover{ filter:brightness (1.2); }</pre>	 <pre>.box{ transition:.3s; } .box:hover{ transform: rotate(18deg); }</pre>	 <pre>.box{ transition:.3s; border:2px solid #ddd; } .box:hover{ border-color: #6366f1; }</pre>	 <pre>.box{ transition:.3s; } .box:hover{ opacity:0.6; }</pre>
---	---	--	--	---

常見可搭配使用的屬性

 transform 移動、縮放、旋轉 (translate / scale / rotate)	 background-color 背景顏色變化	 color 文字顏色變化	 opacity 透明度變化	 box-shadow 陰影變化	 border 邊框變化
--	--	---	--	--	--

小技巧

- ✓ transition 放在預設狀態 (非 :hover) 會更順暢
- ✓ 只設定需要動畫的屬性，避免影響效能 (如：不要 all)
- ✓ 搭配 will-change 屬性可優化效能 (進階用法)

```
.card{
  transition:.3s;
  will-change:transform; /* 優化動畫效能 */
}
```

★ 善用 Transition + Hover，可以讓網站更生動、更有吸引力！🚀

12. Border Radius 外框圓角

13. Box Shadow 陰影

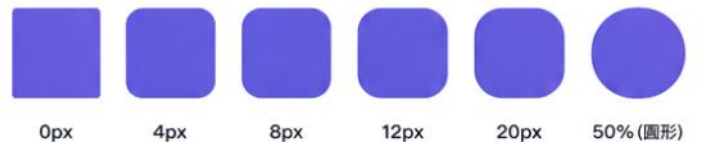
14. Overflow 控制內容是否溢出。

Border Radius

現代網站大量使用圓角，讓設計更柔和、友善。
AI 幾乎每次都會使用。

圖角大小示意圖

```
CSS
.box {
  border-radius: 10px;
}
```



實際應用範例

 卡片設計 常見於卡片、模態框 按鈕等元件。	 立即體驗 按鈕設計 圓角按鈕更顯親和力， 提升使用體驗。	 圖片容器 圖片搭配圓角，視覺 更柔和。	 模態框 圓角讓彈窗看起來 更現代。
--	---	---	--

Box Shadow

增加元素的立體感與層次感，讓畫面更有深度。

陰影組成說明

Box Shadow

增加元素的立體感與層次感，讓畫面更有深度。

陰影組成說明

CSS

```
.card {  
  box-shadow: 0 8px 30px rgba(0,0,0,.2);  
}
```

0

X 偏移

8px

Y 偏移

30px

模糊半徑

rgba(0,0,0,.2)

顏色與透明度

效果對比

無陰影

輕微陰影

0 2px 8px
rgba(0,0,0,.1)

中等陰影

0 8px 20px
rgba(0,0,0,.15)

強烈陰影

0 12px 40px
rgba(0,0,0,.25)

實際應用範例



商品卡片

\$2,680



儲存成功！

資料已離成功儲存。

確定



首頁



分類



購物車



會員

導覽列浮層效果

立即購買

按鈕立體效果

Overflow

控制內容是否溢出容器。

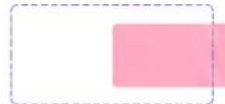
CSS

```
.box-hidden {  
  overflow: hidden;  
}  
.box-auto {  
  overflow: auto;  
}
```

常見屬性值

visible (預設值)

內容超出容器會顯示在外面。



hidden

超出容器的內容會被隱藏。



auto

超出容器的內容會出現捲軸讓使用者滾動。



實際應用範例

overflow: hidden



隱藏超出區域的內容，常用於圖片裁切效果。

overflow: auto

這是一段很長的內容，超過容器高度時，會自動出現捲軸，讓使用者可以捲動瀏覽全部內容。這在聊天視窗、長文章區塊、表格容器等場景非常實用。

15. Position 物件定位

16. CSS 變數

Position

用來控制元素在頁面上的定位方式，
AI 很常使用，建構各種版面效果。

重要屬性

- relative 相對定位（原位置為基準）
- absolute 絕對定位（相對於最近的已定位父層）
- fixed 固定定位（相對於視窗）
- sticky 黏性定位（滾動到指定位置時固定）

固定導覽列效果



導覽列固定在頂部

CSS

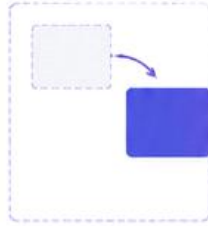
```
header {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  z-index: 1000;  
}  
/* 固定導覽列範例 */
```

定位效果對比

定位效果對比

relative

元素依原本位置，
再進行偏移



absolute

元素脫離文流，
相對於父層定位



fixed

元素脫離文流，
相對於視窗定位



sticky

滾動到指定位置時，
變成固定狀態



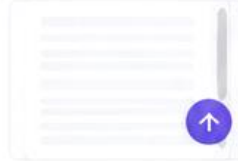
實際應用範例

固定導覽列



使用 position: fixed;
讓導覽列固定在頂部

返回頂部按鈕



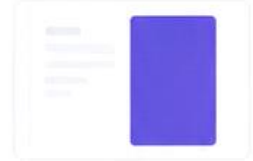
使用 position: fixed;
固定在右下角

彈出視窗 (Modal)



使用 position: fixed;
置中顯示視窗

側邊黏性選單



使用 position: sticky;
滾動到頂部時固定側邊欄

CSS 變數 (★★★★★)

現代 AI 很喜歡產生，方便統一管理顏色、字體、間距等設計變數，
修改一處，全站同步更新！



定義變數

```
CSS
:root {
  --primary: #3b82f6; /* 主要色 */
  --secondary: #10b981; /* 次要色 */
  --dark: #111827; /* 深色文字 */
  --light: #f9fafb; /* 背景色 */
  --radius: 10px; /* 圓角大小 */
  --shadow: 0 8px 30px rgba(0,0,0,.15); /* 陰影 */
  --space: 16px; /* 間距 */
  --font-size: 16px; /* 基礎字體 */
}
```

使用變數

```
CSS
button {
  background: var(--primary);
  color: #fff;
  padding: var(--space) calc(var(--space) * 1.5);
  border: none;
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  font-size: var(--font-size);
  transition: .3s;
}
button:hover {
  background: var(--secondary);
  transform: translateY(-3px);
}
```

實際應用效果

修改前 (--primary: #3b82f6)

主要按鈕 次要按鈕 文字連結



這是一則系統通知訊息
背景色與邊框都使用變數！



只要修改

修改後 (--primary: #8b5cf6)

主要按鈕 次要按鈕 文字連結



這是一則系統通知訊息
背景色與邊框都使用變數！

為什麼要用 CSS 變數？



統一管理
集中管理設計變數，
維護更輕鬆



全站同步
修改一處變數，
全站立即更新



主題切換
輕鬆實現深色/淺色
等主題切換



團隊協作
設計規範一致，
提升開發效率



擴充性高
新增變數方便，
專案更具彈性



變數命名建議使用語意化名稱 (如: --primary, --text-color)

**小技巧：**

- ✓ 變數命名建議使用語意化名稱（如：--primary、--text-color）
- ✓ 將常用變數放在 :root 全域定義，方便重複使用
- ✓ 搭配媒體查詢（Media Query）實現不同裝置的變數值調整

17. AI 常用的 CSS 命名方式

AI 常使用：

container

wrapper

section

hero

navbar

card

card-body

card-title

card-content

feature

service

gallery

pricing

footer

btn

btn-primary

btn-secondary

了解命名方式，可以快速閱讀 AI 生成的程式碼。

二、Flexbox 與 Grid 排版精講

1. Flexbox 彈性排版：一維對齊的魔法

a. 深度核心技術解析

- 當你讓 AI 做一個導覽列、選單或左右排列的按鈕時，AI 經常會讓元件亂跑、黏在一起或上下對齊歪掉。
- 底層物理邏輯：在預設情況下，網頁的標籤都是「固執且由上往下重疊」的（稱為塊級流 Block Stream）。當我們對最外層的大箱子（父層）下達 `display: flex;` 時，這個大箱子會瞬間變成「排隊指導員」。它會強制將內部原本向下堆疊的小箱子（子層），全部轉向水平一字排開（預設的主軸方向 Row）。
- 一定要懂的「彈性調校三大軸心」：
 1. `justify-content`：控制主軸方向（預設為左右水平）的分配。AI 經常不知道怎麼推開元件，而這個屬性可以讓元件「靠左對齊 (`flex-start`)」、「絕對置中 (`center`)」、「靠右對齊 (`flex-end`)」，或是最常用的「兩端均勻推開，中間留空 (`space-between`)」。

- 元件，而這個屬性可以讓元件「靠左對齊 (flex-start)」、「絕對置中 (center)」、「靠右對齊 (flex-end)」，或是最常用的「兩端均勻推開，中間留空 (space-between)」。
2. align-items：控制交叉軸方向（預設為上下垂直）的對齊。網頁裡的 Logo 和文字常常因為字型大小不同而上下對齊歪掉，只要下 align-items: center;，不管小箱子多高，排隊指導員都會強迫它們的中心點維持在同一條水平線上。
 3. gap：控制元件之間的呼吸間距。過去傳統網頁或偷懶的 AI 為了拉開按鈕間距，會私自下 margin-right（右外邊距），這會導致最後一個按鈕的右邊多出一塊空白，排版直接歪掉。現代黃金標準是一律使用 gap，它只會精準地把間距塞在「小箱子與小箱子中間」，前後絕不留贅肉。

b. Prompt 範例對照

模糊 Prompt (導致 AI 亂寫、排版黏在一塊)	「幫我做一個網頁最上面的導覽列，左邊要有 Yuzu Cafe 的名字，右邊要有關於我們跟美味菜單這兩個連結。」
精準 Prompt (強制 AI 使用 Flexbox 彈性佈局)	完美提示詞：「請幫我建立一個網頁頂部的導覽列 (Header)。規範如下： 1. 最外層大外殼必須使用標準的 <header> 語義化標籤，並啟用 Flexbox 彈性排版 (display: flex)。 2. 透過設定 justify-content: space-between; 讓左邊的 Logo 與右邊的導覽選單呈現兩端分散對齊；同時設定 align-items: center; 確保不論高度如何，兩者都在上下垂直方向絕對置中對齊。 3. 右邊的導覽選單 (使用 <nav> 標籤) 內部也必須啟用 Flexbox 排列，並透過 gap: 24px; 屬性設定連結之間的固定安全距離。嚴禁在程式碼中出現 &nbsp; 或傳統 float 浮動語法。」

c. AI生成結果

```

HTML

<header class="navbar">
  <div class="logo">Yuzu Cafe</div>

  <nav class="menu">
    <a href="#about" class="menu-item">關於我們</a>
    <a href="#menu" class="menu-item">美味菜單</a>
  </nav>
</header>

CSS

/* 導覽列外殼：變身排隊指導員 */
.navbar {
  display: flex;
  justify-content: space-between; /* 左右強行推開 */
  align-items: center;           /* 上下絕對置中 */

```

```

display: flex;
justify-content: space-between; /* 左右強行推開 */
align-items: center;           /* 上下絕對置中 */
padding: 20px 40px;
background-color: #ffffff;
box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}

.logo {
font-weight: bold;
font-size: 24px;
color: #111827;
letter-spacing: -0.02em;
}

/* 內層選單：同樣用 flex 橫排，並用 gap 撐開間距 */
.menu {
display: flex;
align-items: center;
gap: 24px; /* 乾淨、安全的黃金呼吸間距 */
}

.menu-item {
text-decoration: none;
color: #4b5563;
font-weight: 500;
font-size: 16px;
transition: color 0.2s;
}

.menu-item:hover {
color: #111827;
}

```

2.Grid 網格排版 —— 二維空間的樂高積木

a. 深度核心技術解析

- Flexbox 雖然強大，但它只擅長處理「單一排（橫的或直的）」的排隊。如果你的版面需要同時處理「橫向的欄」與「縱向的列」（如：商品九宮格、特色牆），就需要出動 Grid。
- 底層物理邏輯：display: grid; 是網頁世界裡的「鋼骨結構九宮格切菜板」。你一旦在最外層的大箱子啟用了 Grid，就等同於在畫布上拉出了一條條無形的水平與垂直坐標軸。你不需要管裡面的元件有多少，食材會自動卡進格子。
- 一定要懂的「無痛切刀語法」：
grid-template-columns: repeat(3, 1fr); $\rightarrow$ repeat(3, ...) 代表連續切出 3 個格子；1fr 代表「等分比例」。最厲害的是，不論你未來的商品卡片文字多寡，Grid 都會強迫同一橫排的卡片高度「集體自動對齊最高的那一張」，徹底根除傳統網頁高低不一的破版災情。

情。

b. Prompt 範例對照

模糊 Prompt (導致 AI 亂寫、欄寬寫死、無法擴充)	「幫我做一個精選商品列表，我要放三個咖啡商品的卡片，幫我排成左右一排。」
精準 Prompt (強制 AI 將 HTML 與 CSS 分流，並啟用 Grid)	完美提示詞：「請幫我做一個 3 欄式的精品咖啡商品列表區塊。規範如下： 1. 最外層大外殼請使用標準的 <section> 標籤，並啟用 CSS Grid 排版 (display: grid)。 2. 請使用 grid-template-columns: repeat(3, 1fr); 建立等寬且具備高度自適應對齊的 3 欄網格佈局。 3. 格子與格子之間的間距請使用 gap: 24px; 控制，不准寫死任何子元件的百分比寬度。 4. 內部的每一個商品卡片請使用 <article> 標籤包裹。 5. 請將 HTML 結構與 CSS 樣式完全分開撰寫，保持程式碼高維護性。」

c. AI生成結果

```
<section class="product-grid">

  <article class="product-card">
    <div class="card-body">
      <h3 class="product-title">衣索比亞 耶加雪菲</h3>
      <p class="product-desc">風味：精緻柑橘、清新檸檬花香，酸質明亮細膩。</p>
    </div>
    <span class="product-price">$350</span>
  </article>

  <article class="product-card">
    <div class="card-body">
      <h3 class="product-title">蘇門答臘 曼特寧</h3>
      <p class="product-desc">風味：濃郁草本、雪松香氣，醇厚度極高，尾韻帶黑巧克力甘甜。</p>
    </div>
    <span class="product-price">$380</span>
  </article>

  <article class="product-card">
    <div class="card-body">
      <h3 class="product-title">巴拿馬 藝妓咖啡</h3>
      <p class="product-desc">風味：夢幻般佛手柑與茉莉花香，層次豐富多變。</p>
    </div>
    <span class="product-price">$500</span>
  </article>

</section>

/* 鋼骨大外殼：切好完美的 3 欄九宮格 */
.product-grid {
```

```

/* 鋼骨大外殼：切好完美的 3 欄九宮格 */
.product-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 均勻平分 3 欄 */
  gap: 24px; /* 格子間距一行搞定，安全不破版 */
  padding: 40px;
  max-width: 1200px;
  margin: 0 auto;
}

/* 卡片衣服：不論文字多長，同一列卡片高度自動強制拉齊！ */
.product-card {
  background-color: #ffffff;
  padding: 20px;
  border-radius: 8px;
  border: 1px solid #e5e7eb;
  display: flex;
  flex-direction: column;
  justify-content: space-between; /* 讓價格內建彈性，永遠沉在卡片最底部 */
}

.product-title {
  margin: 0 0 8px 0;
  color: #111827;
  font-size: 18px;
  font-weight: 700;
}

.product-desc {
  color: #6b7280;
  font-size: 14px;
  margin: 0 0 16px 0;
  line-height: 1.5;
}

.product-price {
  color: #b45309;
  font-weight: 700;
  font-size: 18px;
}

```

三、CSS 變數 —— 一鍵更換全站品牌色的魔法

a. 深度核心技術解析

- 當你讓 AI 盲刷網頁時，它最喜歡把顏色直接死死地寫在幾百個不同的樣式角落（例如按鈕背景寫 #b45309、標題文字寫 #b45309、邊框也寫 #b45309）。當你的專案到了後期，老闆或客戶突然說：「我覺得這個咖啡色不好看，幫我改成優雅的松針綠。」這時你就必須在成千上萬行的程式碼裡瘋狂搜尋、手動一個一個改。這不僅耗時，還極容易漏改。

在成千上萬行的程式碼裡瘋狂搜尋、手動一個一個改。這不僅耗時，還極容易漏改。

- 底層物理邏輯：CSS 變數 (自訂屬性 Custom Properties) 就是為網頁量身打造的「中央控制調色盤」。它的運作原理是：我們在整個網頁的最根部、也就是權重最高的中央倉庫 :root 裡面，開關一個變數油漆桶，並給它取個人類看得懂的名字 (名字前面一定要加上兩條短斜線，例如 --brand-primary) 。
- 一定要懂的「一鍵更換魔法」：
在下方所有的網頁元件衣服上，只要遇到需要上色的地方，一律不准寫死十六進位色碼，必須使用 var(--變數名稱) 進行引用。這樣一來，你的網頁元件本質上只是連接著一條無形的管線回到 :root。當你想換色時，只需要動 :root 倉庫裡那一行色碼，全站幾百個元件會在一微秒內集體變色！

b. Prompt 範例對照

模糊 Prompt (導致 AI 寫死色碼、失去維護性)	「幫我做一個按鈕跟一個標題，按鈕的背景顏色要咖啡色 (#b45309)，標題的文字顏色也要咖啡色 (#b45309) 。」
精準 Prompt (強制 AI 宣告與套用 CSS 變數)	完美提示詞： 「請幫我生成一個包含標題、內文與按鈕的區塊，並全面引入 CSS 原生變數 (Variables) 進行色彩管理： 1. 請在頂層 :root 中央倉庫中宣告三個變數：品牌主打色 (--brand-primary，數值為 #b45309)、文字深色 (--text-dark，數值為 #111827)、背景淺色 (--bg-soft，數值為 #f9fafb)。 2. 底下所有的網頁標籤，其顏色、背景色、文字色，一律不准寫死任何具體色碼，必須強制使用 var(--變數名) 進行調用。 3. 確保樣式結構乾淨，讓我以後只需修改 :root 變數就能一秒切換為其他品牌色系。」

c. AI生成結果

```
<style>
/* 1. 在中央大倉庫宣告油漆桶，貼上好讀的名字標籤 */
:root {
  --brand-primary: #b45309; /* 琥珀咖啡色 */
  --text-dark: #111827; /* 極深灰 ( 比純黑高階 ) */
  --bg-soft: #f9fafb; /* 質感微亮白 */
}

/* 2. 底下衣服全面管線對接 var() */
.content-box {
  padding: 30px;
  background-color: var(--bg-soft);
  border-radius: 12px;
}
```

```

    background-color: var(--bg-soft);
    border-radius: 12px;
  }

  .highlight-title {
    color: var(--brand-primary);
    font-size: 28px;
    font-weight: 800;
    margin: 0 0 12px 0;
  }

  .main-desc {
    color: var(--text-dark);
    font-size: 16px;
    line-height: 1.6;
  }

  .brand-btn {
    background-color: var(--brand-primary);
    color: #ffffff;
    border: none;
    padding: 12px 28px;
    border-radius: 6px;
    font-weight: 600;
    cursor: pointer;
    transition: opacity 0.2s;
  }
  .brand-btn:hover {
    opacity: 0.9;
  }
</style>

<div class="content-box">
  <h2 class="highlight-title">職人嚴選單品豆</h2>
  <p class="main-desc">挑選全球頂級莊園生豆，自家專業新鮮烘焙。</p>
  <button class="brand-btn">立即選購</button>
</div>

```

四、處理 AI 生成後的樣式衝突與優先權混亂

a. 深度核心技術解析

- 在 Vibe Coding 的過程中，學員最常撞牆的災情就是：「我明明聽老師的話，自己寫了一行代碼去改按鈕顏色，但是瀏覽器卻頑固地顯示原本 AI 刷出來的顏色，完全不為所動！」這不是電腦壞掉，而是你遇到了網頁底層的「CSS 權重優先權爭奪戰（Specificity）」。
- 底層優先權計分制（誰喊得越具體，誰就贏）：瀏覽器在解析該給元件穿哪件衣服時，會拿

- 底層優先權計分制（誰喊得越具體，誰就贏）：瀏覽器在解析該給元件穿哪件衣服時，會拿出一張嚴格的計分表：
 1. 內聯樣式（Inline Style，直接寫在 HTML 標籤身上的 `style="..."`） $\rightarrow$ 權重得分：1000 分。它是最強的霸道流氓，可以直接輾壓下方一切在樣式表裡寫的規則。
 2. ID 選擇器（點名特定某個身分證字號，如 `#submit-btn`） $\rightarrow$ 權重得分：100 分。將軍級別。
 3. Class 選擇器（點名某個特徵群組，如 `.btn`） $\rightarrow$ 權重得分：10 分。常規精兵。
 4. 元素選擇器（只喊標籤種類，如 `button`） $\rightarrow$ 權重得分：1 分。最底層平民。
- 抓漏與急救絕招：AI 因為過度依賴框架，經常會寫出像 `html body main section .card div button.btn` 這樣落落長、高達 40 幾分的正規軍樣式，導致你手動寫的 `.btn`（才 10 分）被直接輾壓。此時你有兩大調校急救妙招：
 1. 第一招：直接把你改的衣服寫在標籤身上的 `style="..."` 裡，直接用 1000 分強行輾壓。
 2. 第二招（終極禁忌外掛）：在你的 CSS 屬性值後面加上空一格加上 `!important`。這在瀏覽器裡代表「特赦令」，它會直接凌駕於任何分數計算之上，強行命令瀏覽器執行這行樣式！

b. Prompt 範例對照

模糊 Prompt（導致 AI 給出複雜且無效的 JavaScript 排查，無法解決權重衝突）	「我手動在 <code><style></code> 裡寫了 <code>.btn { background: green; }</code> ，但是按鈕還是顯示黑色的，幫我修一下。」
精準 Prompt（強制 AI 啟用最高優先權覆蓋方案）	完美提示詞：「我手動修改的按鈕顏色被先前複雜的高權重 CSS 樣式覆蓋了，導致網頁畫面完全沒有反應。 1. 請提供我一套最高優先權的樣式覆蓋急救方案。 2. 請在按鈕的 <code>background-color</code> 屬性後方直接附加 <code>!important</code> 語法，或者直接使用 <code>style="..."</code> 內聯樣式 進行覆蓋。 3. 確保該按鈕能強行克服一切外部權重衝突，百分之百在畫面上呈現亮綠色（<code>#059669</code>）。」

c. AI生成結果

```
<style>
/* 假設這是 AI 先前在別的檔案或框架裡生成的超高分數樣式表 */
/* 計分：1個 id(100) + 2個 class(20) + 3個標籤(3) = 123 分！ */
#main-content .product-card div .action-area button.btn {
```

```

/* 計分：1個 id(100) + 2個 class(20) + 3個標籤(3) = 123 分！ */
#main-content .product-card div .action-area button.btn {
  background-color: #000000;
  color: #ffffff;
}

/* 這是你滿懷希望手動追加的代碼。計分：1個 class = 10 分 */
/* 結論：10 分 打不過 123 分，瀏覽器直接無視這行，按鈕頑固維持黑色 */
.btn {
  background-color: green;
}
</style>

<div id="main-content">
  <div class="product-card">
    <div class="action-area">
      <button class="btn" style="padding: 10px 20px;">我是沒變色的悲劇按鈕</button>
    </div>
  </div>
</div>

```

```

<div id="main-content">
  <div class="product-card">
    <div class="action-area">
      <button class="btn" style="background-color: #059669 !important; color: #ffffff !important; border: none; padding: 12px 24px; border-radius: 6px; font-weight: bold; cursor: pointer; box-shadow: 0 2px 4px rgba(0,0,0,0.1);">
        強制覆蓋、整容成功的綠色按鈕
      </button>
    </div>
  </div>
</div>

```

五、RWD 響應式斷點微調 —— 手動修復手機版跑版災情

a. 深度核心技術解析

- 許多小白在 Vibe Coding 時最常遇到的崩潰場景是：網頁在電腦螢幕上看明明非常完美，但只要一把網址傳到手機上打開，元件瞬間被強行擠壓、畫面嚴重歪斜。
- 底層物理邏輯：我們必須手動建立一條防禦邊界，稱為 RWD (響應式網頁設計) 媒體查詢斷點。我們使用 @media (max-width: 768px) 這行指令，它就像是在網頁內部埋設了一個「螢幕寬度量測儀」。它的意思是：當使用者的裝置螢幕總寬度小於或等於 768 像素 (手機與平板的大本營) 時，網頁就會自動觸發警報，強迫裡面的元件換穿我們特製的手機版輕量

「螢幕寬度重測儀」。它的意思是：當使用者的裝置螢幕總寬度小於或等於 768 像素（手機與平板的大本營）時，網頁就會自動觸發警報，強迫裡面的元件換穿我們特製的手機版輕量衣服。

- 一定要懂的「手機防跑版大絕招」：
只要是在電腦版使用了多欄排列的 Grid 佈局，在進入手機版斷點後，手動將其改寫為 `grid-template-columns: 1fr 1important;`。這能一秒解開元件的橫向枷鎖，讓它們大方地在手機螢幕上「上下乖乖單欄直排」，徹底消滅邪惡的橫向捲軸！

b. Prompt 範例對照

模糊 Prompt (導致 AI 忽略行動端、生成無響應式跑版代碼)	「幫我把這三個特色方塊排成左右一排，背景用灰色。」
精準 Prompt (強制 AI 將 HTML 與 CSS 分流，並啟用 RWD 斷點)	完美提示詞：「請幫我做一個 3 欄式的課程特色介紹區塊。規範如下： 1. 電腦版預設：請使用 CSS Grid 佈局，平分成 3 欄，間距為 24px。 2. 必須手動加入標準手機版 RWD 斷點安全機制：請在 CSS 樣式表中寫入 <code>@media (max-width: 768px)</code> 媒體查詢規則。 3. 當螢幕寬度小於 768px 時，利用 <code>grid-template-columns: 1fr 1important;</code> 將網格結構降級為單欄上下直排。 4. 分開編寫 HTML 與 CSS 代碼，確保元件在手機版絕不擠壓跑版。」

c. AI生成結果

```
<section class="responsive-feature-grid">

  <article class="feature-card">
    <h3 class="feature-title">01. 奶泡控制</h3>
    <p class="feature-desc">打出如絲綢般細緻的綿密微奶泡。</p>
  </article>

  <article class="feature-card">
    <h3 class="feature-title">02. 流量手感</h3>
    <p class="feature-desc">精準掌控晃杯幅度與出奶流量黃金比例。</p>
  </article>

  <article class="feature-card">
    <h3 class="feature-title">03. 構圖美學</h3>
    <p class="feature-desc">從經典心型到高難度天鵝對流圖案。</p>
  </article>

</section>
```

```
</section>
```

```
/* =====  
1. 電腦大螢幕：橫向大方平分 3 欄  
===== */  
.responsive-feature-grid {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr); /* 電腦版預設均分 3 欄 */  
  gap: 24px;  
  padding: 40px;  
  max-width: 1200px;  
  margin: 0 auto;  
}  
  
.feature-card {  
  background: #ffffff;  
  padding: 24px;  
  border-radius: 8px;  
  border: 1px solid #e5e7eb;  
  box-shadow: 0 2px 4px rgba(0,0,0,0.02);  
}  
  
.feature-title {  
  color: #b45309;  
  margin: 0 0 8px 0;  
  font-size: 20px;  
  font-weight: 700;  
}  
  
.feature-desc {  
  color: #4b5563;  
  line-height: 1.6;  
  margin: 0;  
  font-size: 15px;  
}  
  
/* =====  
2. 手機響應式防禦塔：當螢幕寬度小於等於 768px 時自動觸發  
===== */  
@media (max-width: 768px) {  
  .responsive-feature-grid {  
    /* 終極解救：強制覆蓋成 1 欄！卡片會自動由上往下直排，寬度完美填滿手機 */  
    grid-template-columns: 1fr !important;  
    padding: 20px;  
  }  
}
```


六、一頁式 Landing Page 視覺美學全面調校綜合實戰

本節為總結。向 AI 下達設計精準級別的「終極防錯 Prompt」，生成一段結構無懈可擊、外觀極具美感、HTML 與 CSS 完全分離的高階一頁式銷售落地頁。

Prompt：

「請幫我建立一個咖啡拉花課程的一頁式 Landing Page 銷售網頁。在編寫時，請嚴格執行以下『美學調校與結構防錯規範』：

1. 色彩管理：必須在頂層 :root 中宣告三個 CSS 變數：深色背景 (#111827)、主打琥珀色 (#b45309)、亮色背景 (#f9fafb)。
2. 一維對齊：網頁頂部 Header 必須使用 Flexbox 佈局，讓 Logo 與報名按鈕呈現兩端對齊，且垂直軸線絕對置中對齊。
3. 二維網格：主內容區的特色介紹必須使用 Grid 佈局，在電腦版預設為均分 3 欄，間距使用 gap: 24px 控制。
4. 手機響應式：必須手動寫入 @media (max-width: 768px) 手機斷點規則。當小於此螢幕寬度時，Grid 網格必須自動覆蓋退化為 1fr 獨立單欄直排。
5. 核心要求：請幫我將 HTML 程式碼與 CSS 程式碼完全切開、分別獨立編寫，禁止使用任何 HTML 內聯樣式。」